ZILO/SYS

Developer's Handbook

FEATURING DAEMON & INDIE

ZILO/SYS Developer Handbook

lsys

April 26, 2026

Contents

1 System Overview **9**
1.1 Architecture . . . 13
1.2 Boot Process . . . 13

2 Bootloader **15**
2.1 Bootloader Stage 1 . . . 19
2.2 Boot Sector Layout . . . 19
2.3 Initialization . . . 19
2.4 Disk Parameter Detection . . . 19
2.5 Root Directory Loading . . . 19
2.6 Stage 2 Lookup . . . 20
2.7 FAT12 Cluster Traversal . . . 20
2.8 Disk Access . . . 20
2.9 LBA to CHS Conversion . . . 20
2.10 Loading Stage 2 . . . 20
2.11 Error Handling . . . 21
2.12 Utility Routines . . . 21
2.12.1 Text Output . . . 21
2.12.2 Disk Reset . . . 21
2.13 Design Constraints . . . 21
2.14 Design Notes . . . 21
2.15 Execution Flow . . . 22
2.16 Future Work . . . 22
2.17 Bootloader Stage 2 . . . 22
2.18 Entry Point . . . 22
2.18.1 Stack Initialization . . . 22
2.19 A20 Line Enable . . . 23
2.20 Global Descriptor Table (GDT) . . . 23
2.21 Switching to Protected Mode . . . 23
2.22 Protected Mode Initialization . . . 24
2.22.1 BSS Initialization . . . 24
2.23 Kernel Loading Pipeline . . . 24
2.24 Disk Abstraction Layer . . . 24
2.25 FAT12 Filesystem . . . 25
2.26 Memory Layout . . . 25
2.27 Memory Detection (E820) . . . 25
2.28 BIOS Real-Mode Interface Layer . . . 26
2.28.1 Mode Switching Macros . . . 26
2.29 BIOS Wrappers . . . 26
2.29.1 Disk Services . . . 26
2.29.2 Memory Services . . . 26

2.29.3 Port I/O . . . 26
2.30 C Runtime Support . . . 26
2.31 Kernel Entry . . . 27
2.32 Design Summary . . . 27
2.33 Future Work . . . 27

3 Kernel Core **29**
3.1 Kernel Entry Point . . . 33
3.2 Entry Function . . . 33
3.3 BSS Initialization . . . 33
3.4 Hardware Abstraction Layer . . . 33
3.5 Virtual Filesystem Initialization . . . 33
3.6 Text Output System . . . 33
3.7 PCI Device Enumeration . . . 34
3.8 Network Stack Initialization . . . 34
3.8.1 ARP Test Request . . . 34
3.9 Block Device Initialization . . . 34
3.10 Filesystem Layer . . . 34
3.11 Framebuffer Interface . . . 34
3.12 Console Banner . . . 35
3.13 Interrupt System (Optional) . . . 35
3.14 System Architecture Overview . . . 35
3.15 Design Notes . . . 35
3.16 Future Work . . . 36
3.17 Kernel Panic System . . . 36
3.17.1 Purpose . . . 36
3.18 Implementation Overview . . . 36
3.19 Interrupt Safety . . . 36
3.20 Visual Output . . . 36
3.21 Register Dump . . . 37
3.21.1 Dumped Registers . . . 37
3.22 System Halt . . . 37
3.23 Simplified Panic Variant . . . 37
3.24 Design Characteristics . . . 37
3.25 Future Work . . . 38
3.26 Timer and PIT . . . 38
3.26.1 Tick-Based Timing . . . 38
3.26.2 Sleep Implementation . . . 38
3.26.3 Requirements . . . 38

4 Memory Management **39**
4.1 Physical Memory Layout . . . 43
4.1.1 Linker Script Structure . . . 43
4.1.2 Memory Map of the Kernel Image . . . 43
4.1.3 BSS Region Handling . . . 43
4.1.4 Kernel Size Determination . . . 44
4.1.5 Design Rationale . . . 44
4.2 Kernel Heap Allocator . . . 44
4.2.1 Heap Layout . . . 44
4.2.2 Allocation Strategy . . . 44
4.2.3 kmalloc Implementation . . . 44
4.2.4 Behavior Characteristics . . . 45

5 Architecture (i686) 47
5.1 Global Descriptor Table (GDT) . . . 51
5.2 Overview . . . 51
5.3 GDT Entry Structure . . . 51
5.3.1 Fields . . . 51
5.4 GDT Layout . . . 51
5.5 Segment Selectors . . . 52
5.6 Loading the GDT . . . 52
5.6.1 Assembly Implementation . . . 52
5.7 Initialization . . . 53
5.8 Design Notes . . . 53
5.9 Future Work . . . 53
5.10 Interrupt Descriptor Table (IDT) . . . 53
5.11 Overview . . . 53
5.12 IDT Entry Structure . . . 53
5.12.1 Fields . . . 54
5.13 IDT Layout . . . 54
5.14 Gate Types . . . 54
5.15 Setting an Entry . . . 54
5.16 Enabling and Disabling Gates . . . 54
5.17 Loading the IDT . . . 55
5.18 Initialization . . . 55
5.19 Descriptor Structure . . . 55
5.20 Design Notes . . . 55
5.21 Relationship with Other Components . . . 55
5.22 Future Work . . . 56
5.23 Interrupt Service Routines (ISR) . . . 56
5.24 Overview . . . 56
5.25 ISR Stubs . . . 56
5.25.1 No Error Code . . . 56
5.25.2 With Error Code . . . 56
5.26 Common ISR Handler . . . 57
5.26.1 Segment Handling . . . 57
5.26.2 Calling the C Handler . . . 57
5.27 Register State Representation . . . 57
5.28 C-Level ISR Handler . . . 57
5.28.1 Handler Dispatch . . . 57
5.29 Exception Handling . . . 58
5.30 Hardware Interrupts . . . 58
5.31 Registering Handlers . . . 58
5.32 Initialization . . . 58
5.33 Design Notes . . . 59
5.34 Future Work . . . 59
5.35 Interrupt Request (IRQ) Handling . . . 59
5.35.1 PIC Remapping . . . 59
5.36 IRQ Handler Dispatching . . . 59
5.36.1 Handler Table . . . 60
5.37 IRQ Initialization . . . 60
5.38 Interrupt Acknowledgement (EOI) . . . 60
5.39 Debugging Support . . . 60
5.40 Design Notes . . . 61

6 Drivers **63**
6.1 ATA Block Device Driver . . . 67
6.1.1 ATA I/O Ports . . . 67
6.1.2 Drive Detection . . . 67
6.1.3 Initialization . . . 67
6.2 Sector Reading (28-bit LBA) . . . 67
6.2.1 Command Setup . . . 68
6.2.2 Status Polling . . . 68
6.2.3 Data Transfer . . . 68
6.3 Sector Writing . . . 68
6.3.1 Write Flow . . . 68
6.3.2 Cache Flush . . . 68
6.4 Timing and Reliability . . . 69
6.5 Error Handling . . . 69
6.6 Design Overview . . . 69
6.7 Future Work . . . 69
6.8 RTL8139 Network Driver . . . 69
6.8.1 PCI Device Initialization . . . 70
6.8.2 Device Reset . . . 70
6.8.3 Buffer Initialization . . . 70
6.8.4 Enabling Transmission and Reception . . . 70
6.8.5 Receive Configuration . . . 70
6.9 MAC Address Retrieval . . . 70
6.10 Packet Transmission . . . 71
6.10.1 Transmission Flow . . . 71
6.11 Register Overview . . . 71
6.12 Future Work . . . 71
6.13 Audio Subsystem . . . 72
6.13.1 PC Speaker Driver . . . 72
6.13.2 Sound Blaster 16 Driver . . . 72
6.13.3 Audio Subsystem Design . . . 74
6.14 Framebuffer Subsystem . . . 74
6.14.1 Framebuffer Structure . . . 75
6.14.2 Initialization . . . 75
6.14.3 Pixel Output . . . 75
6.14.4 Screen Clearing . . . 75
6.15 Framebuffer Terminal . . . 75
6.15.1 Character Rendering . . . 76
6.15.2 Cursor Management . . . 76
6.16 Future Work . . . 76

7 File Systems **77**
7.1 Virtual File System (VFS) . . . 81
7.2 Core Concepts . . . 81
7.3 File Representation . . . 81
7.4 File Operations Interface . . . 81
7.5 File Registration . . . 81
7.6 File Lookup . . . 82
7.7 File I/O Operations . . . 82
7.7.1 Read . . . 82
7.7.2 Write . . . 82

7.7.3 Close . . . 82
7.8 Directory Listing . . . 82
7.9 Design Characteristics . . . 83
7.10 RAM Filesystem (RAMFS) . . . 83
7.11 Design Overview . . . 83
7.12 Inode Structure . . . 83
7.13 File Creation . . . 83
7.14 File Operations . . . 84
7.14.1 Read . . . 84
7.14.2 Write . . . 84
7.14.3 Close . . . 84
7.15 Initialization . . . 84
7.16 Integration with VFS . . . 84
7.17 Future Work . . . 85
7.18 Dataset Filesystem (DSET) . . . 85
7.19 Design Overview . . . 85
7.20 Dataset Structure . . . 85
7.21 Dataset Layout . . . 86
7.21.1 Header Structure . . . 86
7.21.2 File Entry Structure . . . 86
7.22 Initialization . . . 86
7.23 Dataset Management . . . 86
7.23.1 Create Dataset . . . 86
7.23.2 Delete Dataset . . . 87
7.23.3 List Datasets . . . 87
7.24 Raw Dataset Access . . . 87
7.24.1 Write Dataset . . . 87
7.24.2 Read Dataset . . . 87
7.25 File-Based Access . . . 87
7.25.1 Write File . . . 87
7.25.2 Read File . . . 88
7.26 Disk Interaction . . . 88
7.27 Design Characteristics . . . 88
7.28 Future Work . . . 88

8 User Interface 89
8.1 Kernel Shell . . . 93
8.1.1 Command Parsing . . . 93
8.1.2 Execution Model . . . 93
8.1.3 Built-in Commands . . . 93
8.1.4 Filesystem Integration . . . 93
8.1.5 Dataset Management . . . 94
8.1.6 Hardware Interaction . . . 94
8.1.7 Program Execution . . . 94
8.1.8 Output Handling . . . 94
8.1.9 Design Characteristics . . . 94
8.1.10 Future Work . . . 95

9 Built-in Programs **97**
9.1 ZEDIT Text Viewer . . . 101
9.1.1 Execution . . . 101
9.1.2 User Interaction . . . 101
9.1.3 File Loading . . . 101
9.1.4 Display Output . . . 101
9.1.5 Design Characteristics . . . 102
9.1.6 Future Work . . . 102

10 Build System **103**
10.1 Build System . . . 107
10.1.1 Overall Structure . . . 107
10.1.2 Top-Level Build Process . . . 107
10.1.3 Floppy Image Creation . . . 107
10.1.4 Bootloader Build Pipeline . . . 108
10.1.5 Kernel Build Pipeline . . . 108
10.1.6 Linker Scripts . . . 108
10.1.7 Toolchain Integration . . . 108
10.1.8 Utility Tools . . . 109
10.1.9 Directory Structure Awareness . . . 109
10.1.10 Clean Targets . . . 109
10.1.11 Design Summary . . . 109
10.1.12 Future Work . . . 109

11 Reference **111**
11.1 Memory Map . . . 115
11.1.1 Overview . . . 115
.1 Interrupt Descriptor Table (IDT) . . . 115
.1.1 Overview . . . 115
.1.2 Interrupt Vector Map . . . 115
.1.3 Notes . . . 116
.2 Memory Layout . . . 116
.2.1 Overview . . . 116
.2.2 Bootloader Stage 1 . . . 116
.2.3 Bootloader Stage 2 . . . 116
.2.4 Kernel . . . 116
.2.5 Heap and Runtime Memory . . . 116
.3 Boot Timeline . . . 117
.3.1 Boot Sequence Overview . . . 117
.3.2 Stage 1: BIOS Bootloader . . . 117
.3.3 Stage 2: Secondary Bootloader . . . 117
.3.4 Kernel Initialization . . . 117
.3.5 Post-Boot System State . . . 117

Chapter 1

System Overview

ZILO/SYS is a custom operating system targeting the i686 (32-bit x86) architecture.

1.1 Architecture

ZILO/SYS has a monolithic kernel design, where core system components such as memory management, drivers, and filesystem access run in kernel space as of now.

The system is built around a low-level hardware abstraction layer (HAL) that interfaces directly with x86 hardware features, including interrupts, I/O ports, and memory-mapped devices.

Memory management is based on a boot-provided E820 memory map, which is passed from the bootloader to the kernel. This map describes usable and reserved regions of physical memory and is used during early system initialization.

Key design goals include:

- Minimal dependency on external libraries
- Direct hardware access for performance and control
- Simple modular driver architecture
- Early boot visibility into system memory layout

1.2 Boot Process

The boot process begins in BIOS real mode, where the system loads the stage2 bootloader from disk into memory.

Stage2 is responsible for:

- Switching the CPU into protected mode
- Querying system memory using the E820 BIOS interrupt interface
- Building a memory map of usable and reserved regions
- Initializing basic hardware structures required by the kernel

Once initialization is complete, the bootloader prepares a `BootParams` structure containing system information such as the memory map and framebuffer data.

The kernel entry point is then called with the boot parameters passed as arguments. During early kernel startup, the system:

- Clears the BSS section
- Initializes the HAL
- Loads the memory map from boot parameters
- Initializes core subsystems (PCI, drivers, filesystem)

After initialization, the kernel transitions into an interactive shell environment and begins normal operation.

Chapter 2

Bootloader

2.1 Bootloader Stage 1

Stage 1 of the ZILO/SYS bootloader is responsible for initializing the system in real mode and loading the second-stage bootloader from a FAT12-formatted disk.

This code is located in the boot sector (512 bytes) and is loaded by the BIOS at address `0x0000:0x7C00`.

2.2 Boot Sector Layout

The boot sector contains both executable code and a BIOS Parameter Block (BPB), which defines the disk format.

- FAT12 filesystem
- 512 bytes per sector
- 1.44MB floppy disk layout

2.3 Initialization

Execution begins at the label `start`. The bootloader initializes segment registers and the stack:

```
mov ax, 0
mov ds, ax
mov es, ax
mov ss, ax
mov sp, 0x7C00
```

A far return (`retf`) is used to normalize the execution to the expected memory location, which ensures compatibility with BIOS implementations that load the boot sector at different segment:offset combinations.

2.4 Disk Parameter Detection

Instead of relying only on BPB values, the bootloader queries the BIOS for disk geometry:

```
mov ah, 08h
int 13h
```

This retrieves:

- Sectors per track
- Number of heads

2.5 Root Directory Loading

The bootloader calculates the location of the root directory:

- Reserved sectors
- FAT tables

```
ax = reserved + (fat_count * sectors_per_fat)
```

It then computes the size of the root directory and reads it into memory using BIOS interrupt `13h`.

2.6 Stage 2 Lookup

The bootloader searches for the file:

```
STAGE2  BIN
```

Directory entries are 32 bytes each. The search iterates through all entries:

```
repe cmpsb
```

If the file is not found, an error message is displayed and the system waits for user input before rebooting.

2.7 FAT12 Cluster Traversal

Once the file is found, the first cluster is extracted and the FAT table is loaded into memory.

Clusters are read sequentially by following the FAT chain:

- Each FAT12 entry is 12 bits
- Even and odd entries are handled differently

```
cmp ax, 0x0FF8
```

Values above this threshold indicate the end of the cluster chain.

2.8 Disk Access

Disk reads are performed using BIOS interrupt `13h`:

```
mov ah, 02h
int 13h
```

Logical Block Addressing (LBA) is converted to Cylinder-Head-Sector (CHS) format before issuing the read.

A retry mechanism has been added to handle read errors.

2.9 LBA to CHS Conversion

The conversion is implemented as:

```
sector = (LBA % sectors_per_track) + 1
head   = (LBA / sectors_per_track) % heads
cyl    = (LBA / sectors_per_track) / heads
```

2.10 Loading Stage 2

Clusters belonging to `STAGE2.BIN` are loaded into memory at:

```
Segment: 0x0000
Offset:  0x0500
```

After loading completes, execution is transferred:

```
jmp 0x0000:0x0500
```

The BIOS drive number is preserved in register `DL`.

2.11 Error Handling

Errors are handled through simple routines:

- Disk read failure
- File not found

Messages are printed using BIOS interrupt `10h`, and the system waits for a keypress before rebooting.

2.12 Utility Routines

2.12.1 Text Output

Characters are printed using BIOS teletype mode:

```
mov ah, 0x0E
int 0x10
```

2.12.2 Disk Reset

The disk controller can be reset using:

```
mov ah, 0
int 13h
```

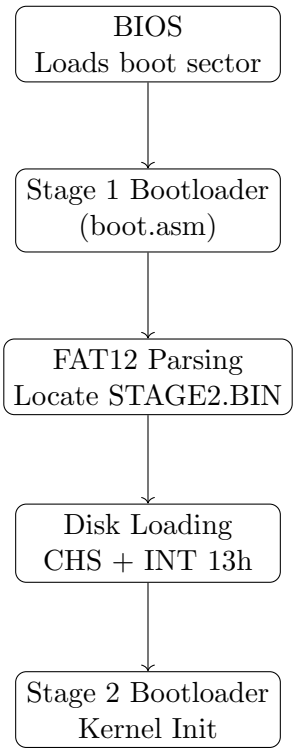
2.13 Design Constraints

- Must fit within 512 bytes
- Executes in 16-bit real mode
- Relies entirely on BIOS services

2.14 Design Notes

- FAT12 is used for simplicity and compatibility
- The bootloader avoids complex abstractions due to size constraints
- Hardcoded values are used where necessary to reduce code size

2.15 Execution Flow



2.16 Future Work

- Support for larger filesystems (FAT16/FAT32)

2.17 Bootloader Stage 2

Stage 2 of the ZILO/SYS bootloader is responsible for transitioning the system from 16-bit real mode into 32-bit protected mode, initializing core runtime services, setting up hardware abstraction layers, loading the kernel from a FAT12 filesystem, and finally transferring execution control to it.

Unlike Stage 1, which is limited to 512 bytes and uses direct BIOS disk access, Stage 2 introduces a structured C-based environment with reusable subsystems including disk I/O, FAT filesystem support, memory detection, and basic hardware access wrappers.

2.18 Entry Point

Execution begins at the symbol `entry`, which is invoked by Stage 1 after loading `STAGE2.BIN`.

```
entry:
    cli
    mov [g_BootDrive], dl
```

The BIOS-provided boot drive number is preserved globally for later disk operations.

2.18.1 Stack Initialization

```
mov ax, ds
mov ss, ax
mov sp, 0xFFF0
mov bp, sp
```

A temporary stack is established within the current segment for early Stage 2 execution.

2.19 A20 Line Enable

The A20 address line is enabled using the keyboard controller to allow access beyond the 1MB boundary.

```
call EnableA20
```

This routine uses direct port I/O via the keyboard controller (`0x60`, `0x64`) to:

- Disable the keyboard
- Read and modify the controller output port
- Enable the A20 line
- Restore keyboard operation

2.20 Global Descriptor Table (GDT)

Stage 2 defines and loads its own Global Descriptor Table:

```
call LoadGDT
lgdt [g_GDTDesc]
```

The GDT includes:

- Null descriptor
- 32-bit kernel code segment
- 32-bit kernel data segment
- 16-bit compatibility code segment
- 16-bit compatibility data segment

The presence of 16-bit segments allows controlled transitions back to real mode for BIOS interrupts.

2.21 Switching to Protected Mode

Protected mode is enabled by setting the PE bit in CR0:

```
mov eax, cr0
or al, 1
mov cr0, eax
```

A far jump is then used to flush the pipeline and load the new code segment:

```
jmp dword 08h:.pmode
```

2.22 Protected Mode Initialization

After entering protected mode:

```
[bits 32]
mov ax, 0x10
mov ds, ax
mov ss, ax
```

Segment registers are reinitialized to the kernel data segment.

2.22.1 BSS Initialization

All uninitialized global data is cleared before entering C code:

```
mov edi, __bss_start
mov ecx, __end
sub ecx, edi
xor al, al
cld
rep stosb
```

2.23 Kernel Loading Pipeline

The kernel is loaded from a FAT12 filesystem stored on disk.

The loading process consists of:

1. Initializing disk geometry via BIOS
2. Reading FAT boot sector
3. Loading FAT table into memory
4. Reading root directory
5. Searching for `/kernel.bin`
6. Following FAT cluster chain
7. Copying file into kernel memory region

The kernel is ultimately loaded at:

```
#define MEMORY_KERNEL_ADDR 0x100000
```

2.24 Disk Abstraction Layer

Stage 2 introduces a structured disk abstraction layer:

- `DISK_Initialize` - retrieves geometry using BIOS INT 13h
- `DISK_ReadSectors` - reads sectors using CHS addressing
- Automatic CHS conversion from LBA

Disk geometry is obtained via:

```
x86_Disk_GetDriveParams
```

This provides:

- Cylinders
- Heads
- Sectors per track

2.25 FAT12 Filesystem

A minimal FAT12 implementation provides file-based kernel loading.

Key structures:

- Boot sector parsing (`FAT_BootSector`)
- Directory entries (`FAT_DirectoryEntry`)
- File abstraction (`FAT_File`)

Cluster-to-LBA conversion:

$$LBA = DataRegionStart + (Cluster - 2) \times SectorsPerCluster \tag{2.1}$$

FAT12 cluster chaining is handled via 12-bit FAT entries.

2.26 Memory Layout

Stage 2 defines a fixed memory map:

- FAT driver: `0x20000`
- Kernel load buffer: `0x30000`
- Kernel execution: `0x100000`

This ensures no overlap between filesystem processing and kernel execution space.

2.27 Memory Detection (E820)

System memory is detected using BIOS interrupt `E820h`:

```
x86_E820GetNextBlock
```

Each returned block is stored as:

```
typedef struct {
    uint64_t Base;
    uint64_t Length;
    uint32_t Type;
    uint32_t ACPI;
} E820MemoryBlock;
```

This produces a full physical memory map used later by the kernel.

2.28 BIOS Real-Mode Interface Layer

Stage 2 uses controlled transitions between protected mode and real mode to access BIOS services.

2.28.1 Mode Switching Macros

- `x86_EnterRealMode`
- `x86_EnterProtectedMode`

These macros:

- Modify CR0 to toggle PE bit
- Perform far jumps to flush CPU pipeline
- Restore segment registers

2.29 BIOS Wrappers

Stage 2 exposes BIOS functionality through protected-mode-safe wrappers.

2.29.1 Disk Services

- `x86_Disk_GetDriveParams`
- `x86_Disk_Read`
- `x86_Disk_Reset`

2.29.2 Memory Services

- `x86_E820GetNextBlock`

2.29.3 Port I/O

- `x86_outb`
- `x86_inb`

These provide low-level hardware access via I/O ports.

2.30 C Runtime Support

Stage 2 includes minimal standard library implementations:

- `memcpy`, `memset`, `memcmp`
- `strlen`, `strcpy`, `strchr`
- `printf` with basic format support
- VGA text mode driver

Video memory is mapped at:

```
0xB8000
```

2.31 Kernel Entry

After initialization and loading, execution is transferred to the kernel:

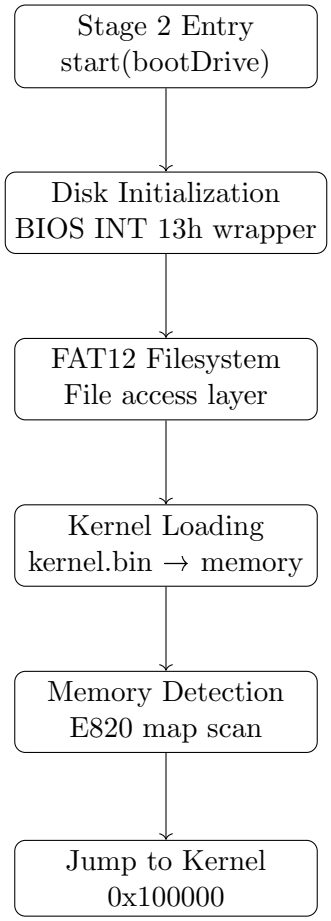
```
KernelStart kernelStart = (KernelStart)Kernel;
kernelStart();
```

At this point, Stage 2 permanently relinquishes control.

2.32 Design Summary

Stage 2 acts as a structured bridge between BIOS and the kernel:

- BIOS access is abstracted through real-mode switching wrappers
- FAT12 filesystem enables file-based kernel loading
- Memory and disk subsystems are initialized before kernel entry
- A minimal C runtime environment is established



2.33 Future Work

- Implement paging before kernel handoff
- Expand filesystem support beyond FAT12

Chapter 3

Kernel Core

3.1 Kernel Entry Point

The ZILO/SYS kernel entry point is responsible for initializing all core subsystems, bringing up hardware drivers, mounting in-memory filesystems, and transitioning the system into a fully operational runtime state.

Unlike the bootloader stages, the kernel operates entirely in 32-bit protected mode with full access to paging, hardware abstraction layers, and driver infrastructure.

3.2 Entry Function

Execution begins at the symbol `start`, placed in a dedicated entry section:

```
void __attribute__((section(".entry")))
start(uint16_t bootDrive, framebuffer_t* fb_info)
```

The bootloader passes:

- `bootDrive` - BIOS disk identifier
- `fb_info` - framebuffer information structure

3.3 BSS Initialization

Before any subsystem is used, the kernel clears its uninitialized data section:

```
memset(&__bss_start, 0, (&__end) - (&__bss_start));
```

This ensures deterministic state for global variables.

3.4 Hardware Abstraction Layer

The first subsystem initialized is the Hardware Abstraction Layer:

```
HAL_Initialize();
```

The HAL provides a unified interface over CPU, interrupt, and platform-specific functionality.

3.5 Virtual Filesystem Initialization

An in-memory filesystem (RAMFS) is initialized early to support kernel data structures:

```
ramfs_init();
```

This filesystem is used for runtime storage of system objects and temporary data.

3.6 Text Output System

Early kernel output is handled through a text rendering subsystem:

```
text_clear();
text_print("Testing ARP...\n");
```

This replaces direct VGA manipulation with an abstraction layer.

3.7 PCI Device Enumeration

The kernel scans the PCI bus to detect hardware devices:

```
pci_scan();
```

This is required for initializing network and storage controllers.

3.8 Network Stack Initialization

The kernel initializes a minimal network stack consisting of:

- RTL8139 network driver
- ARP protocol layer

```
rtl8139_init(0, 3);
arp_init();
```

3.8.1 ARP Test Request

A test ARP request is issued to verify network functionality:

```
arp_request(0xC0A80101); // 192.168.1.1
```

3.9 Block Device Initialization

ATA storage devices are initialized for disk access:

```
ata_init();
ata_read28(0, 1, sector);
```

A test read from LBA 0 is performed to validate disk communication.

3.10 Filesystem Layer

A dataset filesystem layer is initialized:

```
dataset_init();
```

This layer provides structured access to stored kernel data.

3.11 Framebuffer Interface

The kernel receives framebuffer information from the bootloader:

```
framebuffer_t* fb_info
```

This enables future graphical output without relying on VGA text mode.

3.12 Console Banner

After subsystem initialization, the kernel prints the system banner:

```
ZZZZZZZZZZZZZZ IIIIII LL            OOOOOO
ZZ           ZZ   II   LL           OO      OO
           ZZ     II   LL          OO        OO
         ZZ       II   LL          OO        OO
       ZZ         II   LL          OO        OO
  SSSSSSSSSSSS  YYI   LL    YY SSSSSSSSSSSSOO
 SS         ZZSS  IYY  LL  YY  SS  OO      SS
ZSSZZZZZZZZZZ  IIIIIYYLLYYLLLLSS    OOOOOO
 SS                  YYYY     SS
  SSSSSSSSSSSS        YY       SSSSSSSSSSSS
             SS       YY                  SS
             SS       YY                  SS
 SS          SS       YY      SS          SS
  SSSSSSSSSSSS        YY       SSSSSSSSSSSS

ZILO/SYS version 1.4b\
READY
>
```

This indicates successful transition into the interactive kernel state.

3.13 Interrupt System (Optional)

The kernel supports interrupt handlers through the IRQ subsystem:

```
i686_IRQ_RegisterHandler(0, timer);
```

Example timer handler:

```
void timer(Registers* regs)
{
    printf(".");
}
```

3.14 System Architecture Overview

At this stage, the system has fully transitioned into kernel mode:

- Bootloader is no longer used
- BIOS calls are no longer required
- Drivers operate directly on hardware
- Memory is fully managed by the kernel

3.15 Design Notes

- Modular initialization order ensures hardware stability
- Network stack is initialized early for debugging capability
- Storage and filesystem layers are separated for flexibility
- Text subsystem acts as early debug console before GUI exists

3.16 Future Work

- Expand network stack beyond ARP (IP, TCP, UDP)
- Replace ATA with AHCI/NVMe drivers
- Implement full virtual memory paging system
- Add GUI framebuffer compositor
- Introduce Scheduling

3.17 Kernel Panic System

The ZILO/SYS kernel implements a minimal panic handler designed for fatal error reporting and system halt. It is intended to provide immediate diagnostic output while ensuring the system enters a safe non-executing state.

3.17.1 Purpose

A kernel panic is triggered when the system encounters an unrecoverable condition, such as:

- Invalid memory access
- Unexpected CPU state
- Critical subsystem failure
- Explicit developer-triggered crash

3.18 Implementation Overview

The panic system is implemented in C and operates directly on the VGA text buffer through the `text` subsystem.

```
void panic(const char* message, Registers* regs)
```

3.19 Interrupt Safety

Upon invocation, interrupts are immediately disabled to prevent further system state corruption:

```
i686_DisableInterrupts();
```

This ensures the CPU cannot switch context or execute interrupt handlers during the panic state.

3.20 Visual Output

The panic handler sets a high-visibility VGA color scheme:

```
text_set_color(0x4F); // red background, white text
```

A standardized panic header is then printed:

```
!!! KERNEL PANIC !!!
```

Followed by the provided error message.

3.21 Register Dump

If register context is available, the panic handler outputs CPU state information for debugging.

3.21.1 Dumped Registers

The following registers are printed:

- EAX
- EBX
- ECX
- EDX
- EIP
- ESP

Example output format:

```
EAX=XXXXXXXX EBX=XXXXXXXX
ECX=XXXXXXXX EDX=XXXXXXXX
EIP=XXXXXXXX ESP=XXXXXXXX
```

This allows post-mortem debugging of execution state at the time of failure.

3.22 System Halt

After output, the system enters an infinite halt loop:

```
for (;;)
    i686_Panic();
```

The function `i686_Panic()` typically executes the `HLT` instruction, preventing CPU execution while conserving power.

3.23 Simplified Panic Variant

A simplified panic function is also provided:

```
void panic_now(const char* message)
```

This version does not require a register dump and is used for early-stage or unrecoverable internal failures where context is unavailable.

3.24 Design Characteristics

The panic system is intentionally minimal:

- No dynamic memory allocation
- No dependency on scheduler or multitasking
- Direct VGA output only
- Fully deterministic execution path

3.25 Future Work

- Stack backtrace unwinding
- Kernel log buffer persistence

3.26 Timer and PIT

The kernel uses the Programmable Interval Timer (PIT) to generate periodic interrupts on IRQ0.

Each interrupt increments a global tick counter, which forms the basis for timekeeping.

3.26.1 Tick-Based Timing

Time is measured in discrete ticks:

```
uint32_t get_ticks();
void sleep(uint32_t ms);
```

If the PIT is configured to 1000 Hz, each tick represents approximately 1 millisecond.

3.26.2 Sleep Implementation

Sleep is implemented as a busy wait combined with the HLT instruction:

```
while ((pit_get_ticks() - start) < ms)
    asm volatile ("hlt");
```

This reduces CPU usage while waiting for the next timer interrupt.

3.26.3 Requirements

This mechanism requires:

- PIT initialized to known frequency
- IRQ0 enabled in PIC
- Interrupts enabled (STI)

Chapter 4

Memory Management

4.1 Physical Memory Layout

The kernel is loaded at the physical address `0x00100000` (1MB), which is the conventional start of protected-mode free memory on x86 systems.

This is defined in the linker script:

```
phys = 0x00100000;

SECTIONS
{
    . = phys;
    ...
}
```

4.1.1 Linker Script Structure

The kernel is linked as a flat binary image using:

- No ELF headers (OUTPUT_FORMAT(binary))
- Direct physical addressing
- Explicit section placement

4.1.2 Memory Map of the Kernel Image

The kernel image is laid out sequentially in memory:

- `.entry` - early entry code
- `.text` - executable code
- `.data` - initialized global variables
- `.rodata` - read-only constants
- `.bss` - zero-initialized data

Each section has a corresponding symbol exported for runtime use:

```
__entry_start
__text_start
__data_start
__rodata_start
__bss_start
__end
```

4.1.3 BSS Region Handling

The BSS section contains uninitialized global/static variables. It is not stored in the binary image and must be cleared manually at runtime:

```
mov edi, __bss_start
mov ecx, __end
sub ecx, edi
xor al, al
cld
rep stosb
```

This ensures deterministic zero-initialization before C code execution.

4.1.4 Kernel Size Determination

The symbol `__end` marks the end of the loaded kernel image and is used to:

- Determine total kernel size in memory
- Assist in memory allocation boundaries
- Support debugging and diagnostics

4.1.5 Design Rationale

Mapping the kernel at `1MB` avoids:

- BIOS data region (0x000000–0x0009FFFF)
- VGA memory (0x000A0000–0x000BFFFF)
- BIOS ROM area (0x000C0000–0x000FFFFF)

This layout ensures compatibility with legacy PC memory conventions while keeping a large contiguous region for kernel execution.

4.2 Kernel Heap Allocator

The kernel implements a minimal bump-style heap allocator for early-stage dynamic memory management.

At this stage of development, the allocator is designed for simplicity rather than efficiency or safety.

4.2.1 Heap Layout

The heap is defined as a fixed-size static memory region:

```
#define HEAP_SIZE 1024 * 1024
static uint8_t g_heap[HEAP_SIZE];
```

This provides a 1MB contiguous memory pool reserved for kernel allocations.

4.2.2 Allocation Strategy

Memory allocation is performed using a simple linear bump pointer:

```
static size_t g_offset = 0;
```

Each allocation advances the offset forward.

4.2.3 kmalloc Implementation

```
void* kmalloc(size_t size)
{
    if (g_offset + size >= HEAP_SIZE)
        return 0;

    void* ptr = &g_heap[g_offset];
    g_offset += size;

    return ptr;
}
```

4.2.4 Behavior Characteristics

This allocator has the following properties:

- **O(1) allocation time**
- No fragmentation handling
- No free operation (memory is never reclaimed)
- Deterministic and predictable behavior

It is intended to be replaced later by a more advanced allocator supporting:

- Free lists or slab allocation
- Memory alignment guarantees
- Dynamic heap expansion using paging

Chapter 5

Architecture (i686)

5.1 Global Descriptor Table (GDT)

The Global Descriptor Table (GDT) defines the memory segmentation model used by ZILO/SYS on the i686 architecture. It provides the processor with information about memory segments, including their base addresses, limits, and access permissions.

Although modern systems often use paging with a flat memory model, segmentation is still required for correct processor operation in protected mode.

5.2 Overview

Each entry in the GDT describes a memory segment. These segments are referenced through segment selectors stored in CPU registers such as CS, DS, and SS.

ZILO/SYS uses a simplified flat memory model:

- Base address: 0x00000000
- Limit: 0xFFFFFFFF (via 4K granularity)
- Separate code and data segments

5.3 GDT Entry Structure

Each GDT entry is defined as:

```
typedef struct
{
    uint16_t LimitLow;
    uint16_t BaseLow;
    uint8_t  BaseMiddle;
    uint8_t  Access;
    uint8_t  FlagsLimitHi;
    uint8_t  BaseHigh;
} __attribute__((packed)) GDTEntry;
```

5.3.1 Fields

- **Base**: 32-bit starting address of the segment
- **Limit**: Size of the segment
- **Access**: Defines permissions and type
- **Flags**: Granularity and size (16-bit / 32-bit)

5.4 GDT Layout

The GDT used by ZILO/SYS consists of three entries:

1. Null descriptor (required by CPU)
2. Kernel code segment
3. Kernel data segment

```
GDTEntry g_GDT[] = {
    GDT_ENTRY(0, 0, 0, 0),

    GDT_ENTRY(0, 0xFFFFF,
        GDT_ACCESS_PRESENT |
        GDT_ACCESS_RING0 |
        GDT_ACCESS_CODE_SEGMENT |
        GDT_ACCESS_CODE_READABLE,
        GDT_FLAG_32BIT |
        GDT_FLAG_GRANULARITY_4K),

    GDT_ENTRY(0, 0xFFFFF,
        GDT_ACCESS_PRESENT |
        GDT_ACCESS_RING0 |
        GDT_ACCESS_DATA_SEGMENT |
        GDT_ACCESS_DATA_WRITEABLE,
        GDT_FLAG_32BIT |
        GDT_FLAG_GRANULARITY_4K),
};
```

5.5 Segment Selectors

Each GDT entry is referenced using a segment selector:

- Code segment: `0x08`
- Data segment: `0x10`

These values correspond to offsets within the GDT.

5.6 Loading the GDT

The GDT is loaded using the `lgdt` instruction. This is performed by the function:

```
void i686_GDT_Load(GDTDescriptor* descriptor,
                   uint16_t codeSegment,
                   uint16_t dataSegment);
```

5.6.1 Assembly Implementation

```
lgdt [eax]
```

After loading the GDT, a far return (`retf`) is used to reload the code segment register (CS). This is necessary because CS cannot be modified directly.

```
push codeSegment
push .reload_cs
retf
```

Once the code segment is updated, all data segment registers are reloaded:

```
mov ds, ax
mov es, ax
mov fs, ax
mov gs, ax
mov ss, ax
```

5.7 Initialization

The GDT is initialized during early kernel startup:

```
void i686_GDT_Initialize()
{
    i686_GDT_Load(&g_GDTDescriptor,
                  i686_GDT_CODE_SEGMENT,
                  i686_GDT_DATA_SEGMENT);
}
```

5.8 Design Notes

- The system uses a flat memory model for simplicity
- Segmentation is retained only for compatibility with protected mode
- All kernel code runs in Ring 0

5.9 Future Work

- User-mode segments (Ring 3)
- Task State Segment (TSS)
- Per-process memory isolation

5.10 Interrupt Descriptor Table (IDT)

The Interrupt Descriptor Table (IDT) defines how the processor responds to interrupts and exceptions in protected mode. Each entry in the IDT maps an interrupt vector to a handler function.

In ZILO/SYS, the IDT is used to dispatch CPU exceptions, hardware interrupts, and software interrupts to their respective handlers.

5.11 Overview

The IDT contains 256 entries, each corresponding to an interrupt vector (0–255). Each entry specifies:

- Address of the interrupt handler
- Code segment selector
- Gate type and privilege level

5.12 IDT Entry Structure

Each entry in the IDT is defined as:

```
typedef struct
{
    uint16_t BaseLow;
    uint16_t SegmentSelector;
    uint8_t  Reserved;
    uint8_t  Flags;
    uint16_t BaseHigh;
} __attribute__((packed)) IDTEntry;
```

5.12.1 Fields

- **Base**: Address of the interrupt handler
- **SegmentSelector**: Code segment (typically kernel code)
- **Flags**: Gate type, privilege level, and presence

5.13 IDT Layout

ZILO/SYS defines a full IDT with 256 entries:

```
IDTEntry g_IDT[256];
```

Each entry must be initialized before use.

5.14 Gate Types

The IDT supports several gate types:

- 32-bit Interrupt Gate
- 32-bit Trap Gate
- Task Gate

ZILO/SYS primarily uses 32-bit interrupt gates.

5.15 Setting an Entry

Entries are configured using:

```
void i686_IDT_SetGate(int interrupt,
                      void* base,
                      uint16_t segmentDescriptor,
                      uint8_t flags);
```

This function splits the handler address into low and high parts and assigns the appropriate flags.

5.16 Enabling and Disabling Gates

Interrupt entries can be enabled or disabled dynamically:

```
void i686_IDT_EnableGate(int interrupt);
void i686_IDT_DisableGate(int interrupt);
```

These functions modify the `PRESENT` flag of the entry.

5.17 Loading the IDT

The IDT is loaded using the `lidt` instruction:

```
lidt [eax]
```

This is wrapped in the function:

```
void i686_IDT_Load(IDTDescriptor* idtDescriptor);
```

5.18 Initialization

During kernel initialization, the IDT is loaded:

```
void i686_IDT_Initialize()
{
    i686_IDT_Load(&g_IDTDescriptor);
}
```

At this stage, entries must already be configured to avoid undefined behavior when interrupts occur.

5.19 Descriptor Structure

The IDT descriptor is defined as:

```
typedef struct
{
    uint16_t Limit;
    IDTEntry* Ptr;
} __attribute__((packed)) IDTDescriptor;
```

5.20 Design Notes

- The IDT supports 256 interrupt vectors
- All handlers execute in kernel mode (Ring 0)
- Interrupt handling is architecture-specific (i686)

5.21 Relationship with Other Components

The IDT works closely with:

- Interrupt Service Routines (ISR)
- Interrupt Request handling (IRQ)
- Programmable Interrupt Controller (PIC)

These components define how interrupts are generated, routed, and handled.

5.22 Future Work

- User-mode interrupt handling (Ring 3)
- System call interface via software interrupts
- Advanced interrupt routing (APIC)

5.23 Interrupt Service Routines (ISR)

Interrupt Service Routines (ISRs) are responsible for handling CPU exceptions and hardware interrupts. In ZILO/SYS, ISRs provide a unified interface between low-level interrupt handling and higher-level kernel logic.

5.24 Overview

When an interrupt or exception occurs, the CPU automatically pushes the following state onto the stack:

- Instruction pointer (EIP)
- Code segment (CS)
- EFLAGS register
- Stack pointer (ESP) and stack segment (SS) (in privilege transitions)

Some exceptions also push an additional error code.
ZILO/SYS wraps these hardware events into a consistent software interface.

5.25 ISR Stubs

Each interrupt is handled by a small assembly stub. These stubs:

- Normalize the stack layout
- Push the interrupt number
- Jump to a common handler

5.25.1 No Error Code

```
push 0
push interrupt_number
jmp isr_common
```

5.25.2 With Error Code

```
push interrupt_number
jmp isr_common
```

This ensures all interrupts share a consistent stack format.

5.26 Common ISR Handler

All stubs transfer control to a shared routine:

```
isr_common:
    pusha
```

This saves all general-purpose registers.

5.26.1 Segment Handling

The current data segment is saved and replaced with the kernel data segment:

```
mov ax, 0x10
mov ds, ax
mov es, ax
mov fs, ax
mov gs, ax
```

This guarantees that the kernel operates with known segment values.

5.26.2 Calling the C Handler

A pointer to the current stack is passed to the C-level handler:

```
push esp
call i686_ISR_Handler
```

This allows the kernel to access the full CPU state through a structured interface.

5.27 Register State Representation

The CPU state is captured in the following structure:

```
typedef struct
{
    uint32_t ds;
    uint32_t edi, esi, ebp, esp, ebx, edx, ecx, eax;
    uint32_t interrupt;
    uint32_t error;
    uint32_t eip, cs, eflags, useresp, ss;
} Registers;
```

This structure provides a complete snapshot of execution at the time of the interrupt.

5.28 C-Level ISR Handler

The central interrupt dispatcher is:

```
void i686_ISR_Handler(Registers* regs);
```

5.28.1 Handler Dispatch

ZILO/SYS allows dynamic registration of interrupt handlers:

```
ISRHandler g_ISRHandlers[256];
```

When an interrupt occurs:

- If a handler is registered, it is executed
- Otherwise, default behavior is applied

5.29 Exception Handling

CPU exceptions (interrupts ¡ 32) are treated as fatal unless handled.

```
Unhandled exception %d
```

The kernel prints:

- Exception name
- Register state
- Error code

After reporting, the system halts:

```
i686_Panic();
```

5.30 Hardware Interrupts

Interrupts with numbers ≥ 32 are treated as hardware interrupts.

If unhandled:

```
Unhandled interrupt %d
```

5.31 Registering Handlers

Custom handlers can be registered at runtime:

```
void i686_ISR_RegisterHandler(int interrupt,
                              ISRHandler handler);
```

This enables modular driver development.

5.32 Initialization

ISR setup is performed during kernel initialization:

```
void i686_ISR_Initialize()
{
    i686_ISR_InitializeGates();

    for (int i = 0; i < 256; i++)
        i686_IDT_EnableGate(i);

    i686_IDT_DisableGate(0x80);
}
```

5.33 Design Notes

- A unified stack format simplifies handler implementation
- Separation between assembly stubs and C logic improves maintainability
- The system supports up to 256 interrupt vectors

5.34 Future Work

- System call interface via interrupt 0x80
- User-mode interrupt handling
- Interrupt prioritization and masking

5.35 Interrupt Request (IRQ) Handling

The IRQ subsystem provides a high-level abstraction over hardware interrupts generated by the Programmable Interrupt Controller (PIC).

It is responsible for:

- Dispatching hardware interrupts to registered handlers
- Acknowledging interrupts via End-of-Interrupt (EOI)
- Integrating with the CPU interrupt descriptor table (IDT)

5.35.1 PIC Remapping

The legacy PIC uses interrupt vectors that conflict with CPU exceptions (0x00–0x1F). To avoid this, it is remapped:

```
#define PIC_REMAP_OFFSET 0x20
```

This shifts IRQs to the range:

- IRQ0–IRQ7 → 0x20–0x27
- IRQ8–IRQ15 → 0x28–0x2F

5.36 IRQ Handler Dispatching

All IRQs are routed through a single central handler:

```
void i686_IRQ_Handler(Registers* regs)
{
    int irq = regs->interrupt - PIC_REMAP_OFFSET;

    if (g_IRQHandlers[irq] != NULL)
    {
        g_IRQHandlers[irq](regs);
    }

    i686_PIC_SendEndOfInterrupt(irq);
}
```

5.36.1 Handler Table

IRQ handlers are stored in a static dispatch table:

```
IRQHandler g_IRQHandlers[16];
```

Each entry corresponds to one hardware IRQ line.

5.37 IRQ Initialization

The IRQ system is initialized as follows:

```
void i686_IRQ_Initialize()
{
    i686_PIC_Configure(PIC_REMAP_OFFSET, PIC_REMAP_OFFSET + 8);

    for (int i = 0; i < 16; i++)
        i686_ISR_RegisterHandler(PIC_REMAP_OFFSET + i, i686_IRQ_Handler
           );

    i686_EnableInterrupts();
}
```

This process performs:

- PIC remapping to avoid CPU exception overlap
- Registration of a unified ISR entry point for all IRQs
- Enabling CPU interrupts via STI

5.38 Interrupt Acknowledgement (EOI)

After an IRQ is handled, the PIC must be notified so it can issue further interrupts:

```
i686_PIC_SendEndOfInterrupt(irq);
```

Failure to send EOI would block further interrupts from the same line.

5.39 Debugging Support

The handler optionally exposes diagnostic information:

- Interrupt service register (ISR)
- Interrupt request register (IRR)

These help distinguish:

- Currently executing interrupts
- Pending interrupt requests

5.40 Design Notes

The IRQ system is designed as a lightweight dispatch layer:

- Single entry point per IRQ
- Minimal overhead handler lookup
- Compatible with C-level callback registration

This architecture provides a foundation for higher-level drivers such as:

- PIT timer driver (IRQ0)
- Keyboard driver (IRQ1)
- Disk controller interrupts (IRQ14/15)

Chapter 6

Drivers

6.1 ATA Block Device Driver

The ATA driver provides low-level access to IDE-compatible storage devices using Programmed I/O (PIO) mode.

It operates directly against the primary ATA bus and implements 28-bit LBA sector reads and writes.

6.1.1 ATA I/O Ports

The primary ATA controller is mapped to I/O port `0x1F0` with a control register at `0x3F6`.

- `0x1F0` - Data register
- `0x1F1` - Error register
- `0x1F2` - Sector count
- `0x1F3` - LBA low
- `0x1F4` - LBA mid
- `0x1F5` - LBA high
- `0x1F6` - Drive/head register
- `0x1F7` - Status / Command

6.1.2 Drive Detection

The driver performs basic device detection by reading the status register after selecting the master drive:

```
i686_outb(ATA_DRIVE, ATA_DRIVE_MASTER);
uint8_t status = i686_inb(ATA_STATUS);
```

Detection cases:

- `0x00` - No device present
- `0xFF` - Floating bus (no controller)
- Otherwise - ATA device detected

6.1.3 Initialization

The controller is reset using a software soft-reset sequence:

```
i686_outb(ATA_PRIMARY_CTRL, 0x04);
delay;
i686_outb(ATA_PRIMARY_CTRL, 0x00);
```

This ensures the drive is in a known state before issuing commands.

6.2 Sector Reading (28-bit LBA)

The function `ata_read28` implements PIO-based sector reads using 28-bit LBA addressing.

6.2.1 Command Setup

Before issuing a read command:

```
i686_outb(ATA_DRIVE, ATA_DRIVE_MASTER | ((lba >> 24) & 0x0F));
i686_outb(ATA_SECCOUNT, sector_count);
i686_outb(ATA_LBA0, (uint8_t)lba);
i686_outb(ATA_LBA1, (uint8_t)(lba >> 8));
i686_outb(ATA_LBA2, (uint8_t)(lba >> 16));
i686_outb(ATA_COMMAND, ATA_CMD_READ);
```

6.2.2 Status Polling

The driver waits for:

- BSY = 0 (device ready)
- DRQ = 1 (data available)
- ERR = 0 (no error)

Polling is implemented with timeouts to prevent hangs.

6.2.3 Data Transfer

Each sector consists of 512 bytes (256 words):

```
for (int i = 0; i < 256; i++)
    buf[i] = i686_inw(ATA_DATA);
```

6.3 Sector Writing

Writing follows the same command structure as reading, but uses:

```
ATA_CMD_WRITE
```

6.3.1 Write Flow

1. Issue write command
2. Wait for DRQ
3. Transfer 256 words per sector
4. Repeat for all sectors

6.3.2 Cache Flush

After writing, the drive cache is flushed:

```
i686_outb(ATA_COMMAND, ATA_CMD_FLUSH);
```

This ensures data is physically committed to disk.

6.4 Timing and Reliability

The driver enforces strict timing rules:

- 400ns delay after drive select (via status reads)
- Polling loops for BSY and DRQ states
- Timeout protection for all blocking operations

These are required for ATA compliance on real hardware.

6.5 Error Handling

Errors are detected via:

- Status register ERR bit
- Timeout expiration
- BSY stuck condition

On failure, the driver returns `-1` and prints diagnostic messages.

6.6 Design Overview

The ATA driver is designed as a minimal PIO interface:

- No DMA usage
- Direct hardware polling
- Blocking synchronous I/O

This makes it suitable for early boot environments before interrupt-driven storage is available.

6.7 Future Work

- IRQ-based ATA completion handling
- LBA48 support for disks ¿ 128GB
- DMA (Bus Mastering) transfers
- Device abstraction layer for SATA/AHCI migration

6.8 RTL8139 Network Driver

The RTL8139 driver provides basic Ethernet functionality using the Realtek RTL8139 network controller.

It operates via memory-mapped I/O registers obtained through PCI configuration space and implements minimal packet transmission support.

6.8.1 PCI Device Initialization

The driver locates the device using the PCI bus and retrieves its base I/O address:

```
uint32_t bar0 = pci_get_bar0(bus, slot, 0);
io_base = bar0 & ~0x3;
```

The base address is used for all subsequent register access.

6.8.2 Device Reset

The controller is reset using the command register:

```
i686_outb(io_base + RTL_CMD, 0x10);
while (i686_inb(io_base + RTL_CMD) & 0x10);
```

This ensures the device starts in a known state.

6.8.3 Buffer Initialization

Two buffers are allocated for network operations:

- Transmit buffer (TX)
- Receive buffer (RX)

```
static uint8_t tx_buffer[1500];
static uint8_t rx_buffer[8192 + 16 + 1500];
```

The RX buffer is registered with the device:

```
i686_outl(io_base + RTL_RBSTART, (uint32_t)rx_buffer);
```

6.8.4 Enabling Transmission and Reception

The device is enabled by setting the appropriate bits in the command register:

```
i686_outb(io_base + RTL_CMD, 0x0C);
```

6.8.5 Receive Configuration

The receive configuration register (RCR) is set to accept incoming packets:

```
i686_outl(io_base + RTL_RCR, 0xF | (1 << 7));
```

This configuration enables promiscuous mode for debugging purposes.

6.9 MAC Address Retrieval

The network interface MAC address is read directly from the device registers:

```
for (int i = 0; i < 6; i++)
    mac[i] = i686_inb(io_base + i);
```

The driver exposes a helper function:

```
void rtl8139_get_mac(uint8_t out[6]);
```

6.10 Packet Transmission

Packets are transmitted using the first transmit descriptor.

6.10.1 Transmission Flow

1. Copy packet data into the transmit buffer
2. Provide buffer address to the controller
3. Specify packet length
4. Trigger transmission

```
i686_outl(io_base + RTL_TX_ADDR0, (uint32_t)tx_buffer);
i686_outl(io_base + RTL_TX_STATUS0, len);
i686_outb(io_base + RTL_CMD, 0x0C);
```

6.11 Register Overview

Key RTL8139 registers used:

- `RTL_CMD (0x37)` - Command register
- `RTL_RBSTART (0x30)` - RX buffer start
- `RTL_RCR (0x44)` - Receive configuration
- `RTL_TCR (0x40)` - Transmit configuration
- `RTL_TX_ADDR0 (0x20)` - TX buffer address
- `RTL_TX_STATUS0 (0x10)` - TX status / command
- `RTL_ISR (0x3E)` - Interrupt status
- `RTL_IMR (0x3C)` - Interrupt mask

This implementation serves as a foundation for future networking features.

6.12 Future Work

- Interrupt-driven packet reception
- RX ring buffer parsing
- Network stack integration (ARP, IP, TCP/UDP)
- Multiple transmit descriptor support
- DMA-safe memory handling

6.13 Audio Subsystem

The audio subsystem provides low-level sound output through two interfaces:

- PC Speaker (simple tone generation via PIT)
- Sound Blaster 16 (SB16) DSP-based PCM audio output

It is designed as a layered architecture, where simple tone generation is available early in boot, and full PCM playback is enabled when hardware support is detected.

6.13.1 PC Speaker Driver

The PC speaker driver provides basic square-wave tone generation using the Programmable Interval Timer (PIT).

It is primarily used for:

- Boot feedback
- Debug signals
- Simple alerts

Frequency Control

The PIT channel 2 is configured in square-wave mode:

```
uint32_t div = 1193180 / freq;

i686_outb(0x43, 0xB6);
i686_outb(0x42, (uint8_t)(div));
i686_outb(0x42, (uint8_t)(div >> 8));
```

Speaker Enable/Disable

The speaker is controlled via port `0x61`:

```
void speaker_on(uint32_t freq)
{
    pit_prepare(freq);
    i686_outb(0x61, i686_inb(0x61) | 3);
}

void speaker_off()
{
    i686_outb(0x61, i686_inb(0x61) & 0xFC);
}
```

6.13.2 Sound Blaster 16 Driver

The SB16 driver provides PCM audio playback using the Sound Blaster 16 DSP interface.

It operates through legacy ISA I/O ports and supports 16-bit audio playback via DMA.

I/O Port Mapping

- `0x220` - DSP base port (command interface)
- `0x22A` - DSP read port
- `0x22C` - DSP write port
- `0x22E` - DSP status port

Device Initialization

The SB16 is initialized by resetting the DSP and validating its response:

```
i686_outb(0x226, 1);
delay();
i686_outb(0x226, 0);
```

The DSP must respond with `0xAA`:

```
uint8_t val = sb16_read();
if (val != 0xAA)
    return 0;
```

Sample Rate Configuration

Audio playback frequency is configured using the DSP command `0x41`:

```
sb16_write(0x41);
sb16_write(high_byte(rate));
sb16_write(low_byte(rate));
```

Typical values:

- 44100 Hz (CD quality)
- 22050 Hz (low CPU usage)
- 11025 Hz (debug audio)

DMA Audio Buffer

Audio samples are transferred using ISA DMA.

A static buffer is allocated for PCM data:

```
static uint8_t sb16_buffer[65536] __attribute__((aligned(4096)));
```

Important: The buffer must reside below 16MB physical memory for ISA DMA compatibility.

DMA Controller Setup

The DMA controller is programmed to transfer audio samples from memory to the SB16 device.

Steps include:

1. Mask DMA channel
2. Reset flip-flop
3. Set address and page register
4. Set transfer length
5. Unmask DMA channel

Playback Command

PCM playback is started using the SB16 DSP command:

```
sb16_write(0xB6); // 16-bit auto-init playback
sb16_write(mode);
sb16_write(length_low);
sb16_write(length_high);
```

Interrupt Handling

Upon completion of playback, the SB16 triggers an IRQ (typically IRQ 5 or IRQ 7).

The interrupt handler is responsible for:

- Acknowledging the interrupt
- Clearing DSP status
- Preparing next audio buffer (if streaming)

Driver Status

Current implementation status:

- DSP detection: **Complete**
- Sample rate configuration: **In progress**
- DMA playback: **Not implemented**
- IRQ streaming: **Not implemented**

6.13.3 Audio Subsystem Design

The audio system is designed with a layered abstraction:

- **Level 1:** PC Speaker (early boot support)
- **Level 2:** SB16 initialization (hardware detection)
- **Level 3:** PCM playback via DMA
- **Level 4:** Streaming audio engine (future work)

6.14 Framebuffer Subsystem

The framebuffer subsystem provides direct pixel-based rendering by writing to a linear memory buffer exposed by the bootloader or firmware.

It enables basic graphics output without relying on legacy text mode.

6.14.1 Framebuffer Structure

The framebuffer is described by the following structure:

```
typedef struct {
    uint32_t* addr;
    uint32_t width;
    uint32_t height;
    uint32_t pitch;
    uint8_t bpp;
} framebuffer_t;
```

Fields:

- `addr` - base address of the framebuffer
- `width` - horizontal resolution (pixels)
- `height` - vertical resolution (pixels)
- `pitch` - number of bytes per scanline
- `bpp` - bits per pixel

6.14.2 Initialization

```
void fb_init(framebuffer_t* fb);
```

Stores framebuffer parameters for global use within the kernel.

6.14.3 Pixel Output

```
void fb_putpixel(int x, int y, uint32_t color);
```

Writes a single pixel to the framebuffer.
The address calculation is:

$$\text{pixel} = \text{addr} + (y \times \text{pitch}) + (x \times \text{bytesPerPixel}) \tag{6.1}$$

Bounds checking is performed to prevent invalid memory access.

6.14.4 Screen Clearing

```
void fb_clear(uint32_t color);
```

Fills the entire framebuffer with a single color by iterating over all pixels.

6.15 Framebuffer Terminal

A minimal terminal layer is implemented on top of the framebuffer.

Instead of text rendering via fonts, the current implementation draws fixed-size blocks to represent characters.

6.15.1 Character Rendering

```
void fb_terminal_putc(char c);
```

Each character is rendered as a solid block:

- Character width: 8 pixels
- Character height: 16 pixels

Rendering is performed by filling a rectangular region using repeated calls to `fb_putpixel`.

6.15.2 Cursor Management

The terminal maintains internal cursor coordinates:

- `cursor_x`
- `cursor_y`

After each character:

- `cursor_x` increments
- Wrap occurs at 80 columns
- `cursor_y` advances on overflow

6.16 Future Work

- Implement bitmap font rendering
- Add scrolling and line buffering
- Support variable resolutions and dynamic layouts
- Introduce double buffering to reduce flicker
- Audio buffer queueing system
- Interrupt-driven streaming playback
- Unified `audio_play()` abstraction layer

Chapter 7

File Systems

7.1 Virtual File System (VFS)

The Virtual File System (VFS) provides a unified interface for interacting with files, abstracting underlying storage implementations such as FAT, memory-backed files, or device nodes.

It allows the kernel to operate on files without needing to know the specific filesystem or driver implementation.

7.2 Core Concepts

The VFS is built around three main components:

- File descriptors (`vfs_file_t`)
- File operations (`vfs_ops_t`)
- A global file registry

7.3 File Representation

```
struct vfs_file
{
    const char* name;
    void* internal;
    const vfs_ops_t* ops;
};
```

Fields:

- `name` - file identifier (used for lookup)
- `internal` - filesystem-specific data
- `ops` - pointer to file operation handlers

7.4 File Operations Interface

```
typedef struct vfs_ops
{
    int (*read)(vfs_file_t* file, void* buf, size_t size, size_t offset
        );
    int (*write)(vfs_file_t* file, const void* buf, size_t size, size_t
        offset);
    int (*close)(vfs_file_t* file);
} vfs_ops_t;
```

Each file provides its own implementation of these operations, allowing polymorphic behavior across different file types.

7.5 File Registration

```
void vfs_register_file(vfs_file_t* file);
```

Files are registered into a global table:

- Maximum of 64 files
- Stored in a static array
- No dynamic allocation required

7.6 File Lookup

```
vfs_file_t* vfs_open(const char* path);
```

File lookup is performed by:

- Iterating through registered files
- Comparing names using `strcmp`

If a match is found, a pointer to the file structure is returned.

7.7 File I/O Operations

7.7.1 Read

```
int vfs_read(vfs_file_t* file, void* buf, size_t size, size_t offset);
```

Delegates to the file's `read` handler.

7.7.2 Write

```
int vfs_write(vfs_file_t* file, const void* buf, size_t size, size_t
    offset);
```

Delegates to the file's `write` handler.

7.7.3 Close

```
int vfs_close(vfs_file_t* file);
```

Invokes the file-specific cleanup routine.
All operations validate the presence of handlers before invocation.

7.8 Directory Listing

```
void vfs_list(vfs_list_callback_t cb);
```

Iterates over all registered files and invokes a callback function:

- Enables simple directory listing
- Decouples output logic from VFS

7.9 Design Characteristics

- Extremely simple and static design
- No hierarchical directory structure
- No dynamic allocation or reference counting
- Function-pointer based polymorphism

7.10 RAM Filesystem (RAMFS)

The RAM Filesystem (RAMFS) is a simple in-memory filesystem implementation built on top of the VFS layer.

It stores file contents directly in dynamically allocated memory, providing fast access without requiring any block device.

7.11 Design Overview

RAMFS integrates with the VFS by:

- Creating file objects (`vfs_file_t`)
- Attaching filesystem-specific data (inodes)
- Providing operation handlers via `vfs_ops_t`

Each RAMFS file is fully resident in memory.

7.12 Inode Structure

```
typedef struct
{
    char* data;
    size_t size;
} ramfs_inode_t;
```

Fields:

- `data` - pointer to file contents
- `size` - size of the file in bytes

7.13 File Creation

```
vfs_file_t* ramfs_create_file(const char* name, const char* content);
```

File creation involves:

1. Allocating an inode structure using `kmalloc`
2. Allocating memory for file contents
3. Copying the provided string into the data buffer
4. Creating a VFS file object
5. Assigning RAMFS operation handlers

7.14 File Operations

RAMFS implements the standard VFS interface.

7.14.1 Read

```
int ramfs_read(vfs_file_t* file, void* buf, size_t size, size_t offset)
    ;
```

Behavior:

- Validates offset against file size
- Clamps read size to remaining data
- Copies data into the provided buffer

7.14.2 Write

```
int ramfs_write(vfs_file_t* file, const void* buf, size_t size, size_t
    offset);
```

Behavior:

- Only allows writes within existing file bounds
- Does not support dynamic resizing
- Returns error if write exceeds allocated size

7.14.3 Close

```
int ramfs_close(vfs_file_t* file);
```

Currently performs no operation and always succeeds.

7.15 Initialization

```
void ramfs_init();
```

Initializes the RAMFS by registering predefined files:

- `hello.txt`
- `readme.txt`

These files are created in memory and added to the VFS registry.

7.16 Integration with VFS

RAMFS relies entirely on the VFS for:

- File lookup
- Operation dispatch
- File listing

From the kernel's perspective, RAMFS files behave like any other VFS file.

7.17 Future Work

- Support dynamic file resizing
- Implement directory structures
- Add file deletion and memory reclamation
- Integrate with a mount system
- Use RAMFS as a potential initial root filesystem

7.18 Dataset Filesystem (DSET)

The Dataset Filesystem (DSET) is a simple block-based storage system built directly on top of the ATA driver.

It provides a lightweight method for storing named data regions ("datasets") on disk, each of which can optionally contain multiple files.

Unlike traditional filesystems, DSET uses a flat allocation model with minimal metadata and no hierarchical structure.

7.19 Design Overview

The system is composed of:

- A global dataset table stored on disk
- Individual datasets mapped to contiguous LBA ranges
- Optional file tables within each dataset

Datasets are identified by name and mapped to physical disk sectors.

7.20 Dataset Structure

```
typedef struct {
    char name[DATASET_NAME_LEN];
    uint32_t start_lba;
    uint32_t size;
} dataset_t;
```

Fields:

- `name` - dataset identifier
- `start_lba` - starting sector on disk
- `size` - size in sectors

The dataset table is stored at a fixed location:

- LBA 1 - dataset table (persistent metadata)

7.21 Dataset Layout

Each dataset occupies a contiguous region on disk:

- Sector 0 - dataset header
- Sector 1 - file table
- Sector 2+ - file data

7.21.1 Header Structure

```
typedef struct {
    uint32_t magic;
    uint32_t file_count;
} dset_header_t;
```

The magic value (`0x3153445A`) identifies a valid dataset.

7.21.2 File Entry Structure

```
typedef struct {
    char name[DSET_FILENAME_LEN];
    uint32_t offset;
    uint32_t size;
} dset_file_t;
```

Each entry describes:

- File name
- Offset within dataset (bytes)
- File size

7.22 Initialization

```
void dataset_init();
```

Loads the dataset table from disk into memory.

7.23 Dataset Management

7.23.1 Create Dataset

```
int dataset_create(const char* name, uint32_t size);
```

Behavior:

- Finds a free slot in the dataset table
- Allocates space after the last used dataset
- Stores metadata and persists it to disk

7.23.2 Delete Dataset

```
int dataset_delete(const char* name);
```

Clears the dataset entry from the table.
Note: Underlying disk data is not erased.

7.23.3 List Datasets

```
void dataset_list();
```

Displays all registered datasets with:

- Name
- Starting LBA
- Size

7.24 Raw Dataset Access

7.24.1 Write Dataset

```
int dataset_write(const char* name, const uint8_t* data, uint32_t size)
    ;
```

Writes raw data directly to the dataset region.

7.24.2 Read Dataset

```
int dataset_read(const char* name, uint8_t* buffer);
```

Reads the entire dataset into memory.

7.25 File-Based Access

DSET also supports a simple file abstraction within each dataset.

7.25.1 Write File

```
int dset_write_file(const char* dset,
                    const char* fname,
                    const uint8_t* data,
                    uint32_t size);
```

Process:

1. Locate dataset
2. Initialize header if necessary
3. Load file table
4. Find existing entry or free slot
5. Append data to dataset region
6. Update metadata and persist changes

7.25.2 Read File

```
int dset_read_file(const char* dset,
                   const char* fname,
                   uint8_t* buffer);
```

Reads file data by:

- Searching file table
- Calculating LBA offset
- Reading required sectors via ATA

7.26 Disk Interaction

All disk access is performed through the ATA driver:

- `ata_read28`
- `ata_write28`

This allows direct sector-level control without intermediate caching layers.

7.27 Design Characteristics

- Extremely simple layout
- Contiguous allocation per dataset
- Minimal metadata overhead
- Direct block-level access

7.28 Future Work

- Integrate DSET with VFS as a mountable filesystem
- Add dynamic allocation and fragmentation handling
- Implement caching and buffering
- Add data integrity checks (checksums)

Chapter 8

User Interface

8.1 Kernel Shell

ZILO/SYS includes a minimal interactive shell that provides a command-line interface for interacting with kernel subsystems such as the VFS, ATA driver, PCI bus, and dataset filesystem.

The shell acts as a lightweight user interface layer, primarily intended for debugging, testing, and basic system interaction.

8.1.1 Command Parsing

Input is parsed into two components:

- Command name
- Argument string

Parsing is performed using simple string utilities:

```
get_command(cmdline, cmd);
const char* args = get_args(cmdline);
```

Whitespace is skipped, and commands are limited to a fixed-size buffer.

8.1.2 Execution Model

Commands are dispatched using a linear string comparison chain:

```
if (strcmp(cmd, "help") == 0) { ... }
else if (strcmp(cmd, "cat") == 0) { ... }
...
```

Each command executes synchronously and returns a status code of type `retc_t`.

8.1.3 Built-in Commands

The shell provides several built-in commands:

- `help` - Displays available commands
- `cat <file>` - Reads and prints a file from the VFS
- `clear` - Clears the screen
- `crash` - Triggers a kernel panic
- `hello` - Prints a test message
- `say <text>` - Prints user-provided text

8.1.4 Filesystem Integration

The shell interfaces with the Virtual File System:

```
vfs_file_t* f = vfs_open(args);
vfs_read(f, buf, size, 0);
```

This allows commands such as `cat` and `listdir` to operate on abstract file objects independent of the underlying filesystem.

8.1.5 Dataset Management

Dataset filesystem operations are exposed via shell commands:

- `createdset <name>` - Creates a dataset
- `deldset <name>` - Deletes a dataset
- `listdset` - Lists all datasets

These commands interface directly with the dataset subsystem:

```
dataset_create(args, 10);
dataset_delete(args);
dataset_list();
```

8.1.6 Hardware Interaction

The shell provides direct access to hardware-level diagnostics:

- `listdisk` - Detects ATA devices
- `listpci` - Scans PCI bus

Example:

```
if (ata_detect()) { ... }
pci_scan();
```

8.1.7 Program Execution

The shell can invoke kernel-integrated programs such as:

- `zedit` - Text editor

```
zedit_run(args);
```

This establishes a foundation for a future executable or userspace model.

8.1.8 Output Handling

All output is routed through the kernel text subsystem:

```
text_print("Hello\n");
```

This ensures consistent rendering regardless of the underlying display backend.

8.1.9 Design Characteristics

- Minimal and synchronous execution model
- No process isolation or multitasking
- Direct coupling with kernel subsystems
- Fixed command set with static dispatch

8.1.10 Future Work

- Command table instead of chained comparisons
- Argument parsing with quoting and flags
- Script execution support
- Integration with a userspace environment
- Process and job control

Chapter 9

Built-in Programs

9.1 ZEDIT Text Viewer

ZEDIT is a minimal text viewing utility integrated into the ZILO/SYS kernel. It provides a simple interface for accessing and displaying files stored within the dataset filesystem.

Although currently limited to read-only functionality, ZEDIT establishes the foundation for future interactive text editing capabilities.

9.1.1 Execution

ZEDIT is invoked from the shell:

```
zedit <args>
```

Internally, the shell dispatches execution via:

```
zedit_run(args);
```

9.1.2 User Interaction

ZEDIT operates interactively by prompting the user for:

- Dataset name
- File name

Input is collected using the keyboard subsystem:

```
char* line = keyboard_get_line();
```

This input is copied into fixed-size buffers for processing.

9.1.3 File Loading

Files are retrieved from the dataset filesystem:

```
dset_read_file(dset, fname, buffer);
```

The file contents are loaded into a static buffer:

```
#define MAX_BUF 4096
static char buffer[MAX_BUF];
```

If the file does not exist, an error message is displayed.

9.1.4 Display Output

Once loaded, the file contents are printed to the screen:

```
text_print(buffer);
```

The output is preceded by a simple header:

```
=== ZEDIT ===
```

9.1.5 Design Characteristics

- Static buffer-based implementation
- Read-only file access
- Blocking, line-based user input
- Tight integration with dataset filesystem

9.1.6 Future Work

- Full text editing (insert/delete operations)
- Cursor movement and screen navigation
- File saving via dataset write API
- Syntax highlighting or structured display

Chapter 10

Build System

10.1 Build System

The ZILO/SYS build system is a multi-stage GNU Make-based pipeline responsible for compiling, linking, and packaging the operating system into a bootable disk image.

It is designed to separate concerns between bootloader stages, kernel compilation, tooling, and final image generation.

10.1.1 Overall Structure

The build system is divided into the following components:

- Stage 1 Bootloader (16-bit real mode, raw binary)
- Stage 2 Bootloader (32-bit protected mode, ELF-linked binary)
- Kernel (freestanding 32-bit binary)
- Tools (host-side utilities such as FAT tooling)
- Floppy image generator

10.1.2 Top-Level Build Process

The root Makefile orchestrates the full build:

```
all: floppy_image tools_fat
```

The final output is a bootable FAT12 floppy disk image:

```
$(BUILD_DIR)/main_floppy.img
```

10.1.3 Floppy Image Creation

The disk image is generated using standard UNIX utilities:

- `dd` — creates a blank 1.44MB floppy image
- `mkfs.fat` — formats the image as FAT12
- `mcopy` — writes files into the FAT filesystem
- `mmd` — creates directories inside the image

```
dd if=/dev/zero of=main_floppy.img bs=512 count=2880
mkfs.fat -F 12 -n "NBOS" main_floppy.img
```

Boot components are then injected:

- `stage1.bin` written to boot sector
- `stage2.bin` copied into filesystem root
- `kernel.bin` copied into filesystem root

10.1.4 Bootloader Build Pipeline

Stage 1

Stage 1 is a 512-byte boot sector written in assembly:

```
nasm boot.asm -f bin -o stage1.bin
```

It produces a flat binary with no linking stage.

Stage 2

Stage 2 is a 32-bit protected mode binary compiled from C and assembly:

```
$(TARGET_CC) -ffreestanding -nostdlib -c
$(TARGET_LD) -T linker.ld -nostdlib
```

It is linked at physical address:

```
phys = 0x00000500;
```

10.1.5 Kernel Build Pipeline

The kernel is compiled as a freestanding binary with full C source recursion support:

- Recursive source discovery via `wildcard`
- Separate compilation of C and ASM objects
- Link-time memory layout defined by linker script

Kernel load address:

```
phys = 0x00100000;
```

10.1.6 Linker Scripts

Both Stage 2 and Kernel use custom linker scripts defining memory layout:

- Entry point symbol (`entry` or `start`)
- Section mapping (.text, .data, .bss, .rodata)
- Flat binary output format

10.1.7 Toolchain Integration

The build system relies on a custom cross-toolchain configured via:

- `TARGET_CC`
- `TARGET_LD`
- `TARGET_ASM`

All kernel and boot components are built using freestanding flags:

```
-ffreestanding -nostdlib -lgcc
```

10.1.8 Utility Tools

A host-side FAT utility is built separately under `tools/fat`:

- Used for filesystem manipulation
- Compiled with host GCC
- Outputs debugging and filesystem tooling binaries

10.1.9 Directory Structure Awareness

The build system automatically handles nested source trees:

```
$(wildcard */*.c)
$(wildcard */*/*.c)
$(wildcard */*/*/*.c)
```

This allows modular expansion without modifying build scripts.

10.1.10 Clean Targets

Each subsystem defines independent cleanup rules:

- Bootloader stage 1 cleanup
- Bootloader stage 2 cleanup
- Kernel cleanup
- Full workspace cleanup

10.1.11 Design Summary

- Fully Makefile-driven build pipeline
- Separation of bootloader, kernel, and tools
- FAT12 disk image as final deployment target
- Freestanding cross-compiled environment
- Modular recursive source discovery

10.1.12 Future Work

- Incremental builds with dependency tracking
- CMake or Ninja backend support

Chapter 11

Reference

11.1 Memory Map

11.1.1 Overview

The system uses a flat physical memory model without paging. All components are placed at fixed addresses defined by bootloader and linker scripts.

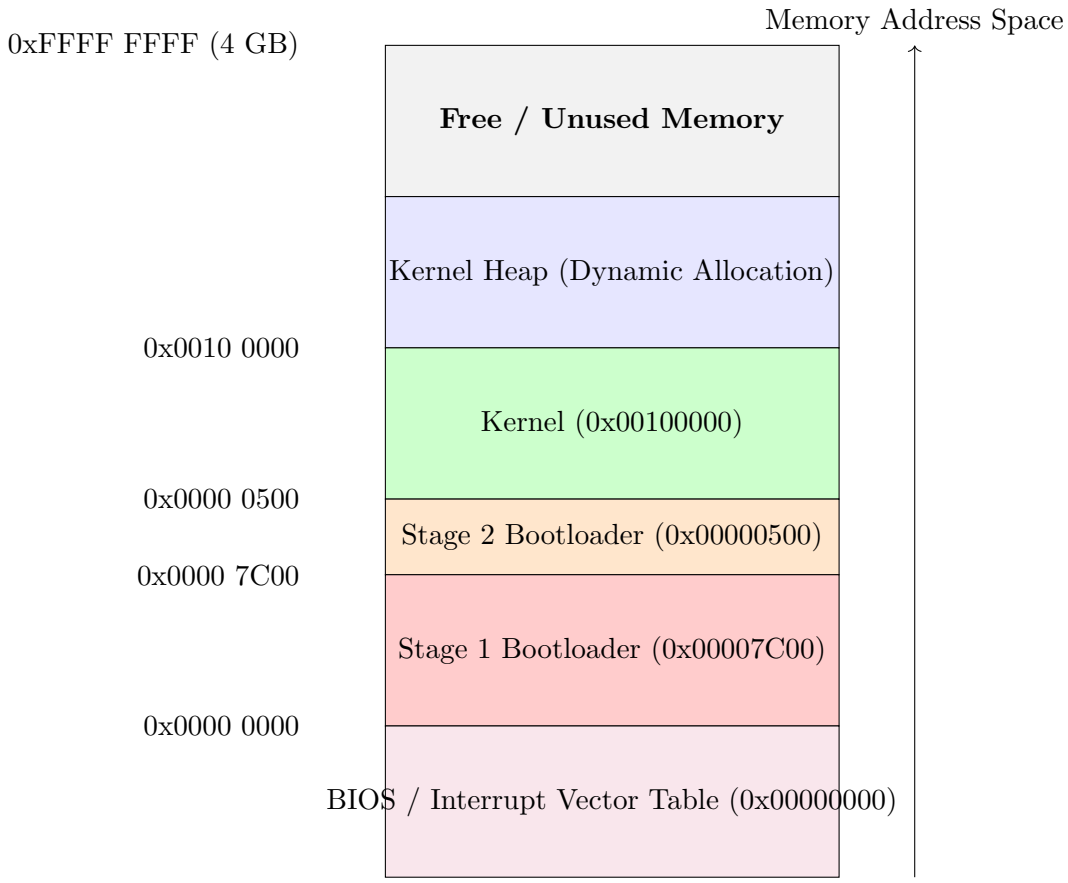


Figure 11.1: Physical Memory Layout of the System

.1 Interrupt Descriptor Table (IDT)

.1.1 Overview

The i686 IDT maps 256 interrupt vectors (0–255) to their corresponding ISR handlers. Vectors 0–31 are CPU exceptions, 32–47 are hardware IRQs (PIC-mapped), and 48–255 are available for software/system use.

.1.2 Interrupt Vector Map

Vector Range	ISR Handler	Description
0–31	i686_ISR0 – i686_ISR31	CPU Exceptions (x86 faults/traps)
32–47	i686_ISR32 – i686_ISR47	Hardware IRQs (PIC remapped interrupts)
48–255	i686_ISR48 – i686_ISR255	Software / user-defined interrupts

Table 1: i686 Interrupt Vector Assignment Summary

.1.3 Notes

- Each ISR is installed via `i686_IDT_SetGate(vector, handler, code_segment, flags)`.
- All entries use a 32-bit interrupt gate with ring 0 privileges.
- Hardware IRQs are typically remapped to avoid overlapping CPU exception vectors.

.2 Memory Layout

.2.1 Overview

The system uses a flat memory model with physical addressing defined by the linker scripts in both the bootloader and kernel stages. Each stage is placed at a fixed physical address to ensure predictable loading without relying on a higher-level loader.

.2.2 Bootloader Stage 1

- **Start Address:** 0x0000:0x7C00 (BIOS load address)
- **Size:** 512 bytes (boot sector)
- **Role:** Loads Stage 2 from disk

.2.3 Bootloader Stage 2

- **Load Address:** 0x00000500
- **Format:** Flat binary (no relocation)
- **Role:** Initializes protected mode and loads kernel

.2.4 Kernel

- **Load Address:** 0x00100000 (1 MB mark)
- **Format:** Flat binary (linked via custom linker script)
- **Sections:**
  - `.entry` – Entry point
  - `.text` – Code segment
  - `.data` – Initialized data
  - `.rodata` – Read-only data
  - `.bss` – Uninitialized memory

.2.5 Heap and Runtime Memory

- Dynamically allocated memory is managed by the kernel heap (if enabled).
- Memory above kernel end (`__end`) is considered free for allocation.
- No virtual memory paging is used in the current implementation.

.3 Boot Timeline

.3.1 Boot Sequence Overview

The system follows a staged boot process starting from BIOS execution to kernel initialization.

.3.2 Stage 1: BIOS Bootloader

- BIOS loads the first 512-byte boot sector into memory at 0x7C00.
- Stage 1 initializes minimal environment.
- Stage 1 loads Stage 2 from disk into memory.

.3.3 Stage 2: Secondary Bootloader

- Loaded at physical address 0x00000500.
- Switches CPU into 32-bit protected mode.
- Sets up GDT (Global Descriptor Table).
- Loads kernel binary from FAT12 filesystem.
- Transfers control to kernel entry point.

.3.4 Kernel Initialization

- Kernel is loaded at 0x00100000.
- Initializes IDT (Interrupt Descriptor Table).
- Installs ISR handlers (0–255 vector table).
- Initializes basic drivers (keyboard, memory, filesystem).
- Enters main kernel loop or scheduler (if implemented).

.3.5 Post-Boot System State

- Interrupts enabled via IDT.
- Protected mode active.
- FAT12 filesystem mounted from floppy image.
- System ready for user commands or shell execution.

Index

A20, 22
ATA, 32

bootloader
    entry, 22
    stage1, 19
    stage2, 22
build system, 105

CHS, 20

disk, 24
driver
    interrupts, 56

exception, 56

FAT12, 24
filesystem, 32
    dataset, 82
framebuffer, 32, 70

GDT, 22, 49

HAL, 31

IDT, 51
interrupt
    stack layout, 55
interrupt gate, 52
interrupts
    IRQ, 57
IRQ, 54
ISR, 53, 54

kernel
    entry, 31
    panic, 34
kernel loading, 23

LBA, 20

MAC address, 68
memory, 24, 25
    heap, 42
memory initialization, 31
memory layout, 41

network, 32
    RTL8139, 67
    transmit, 69

PCI, 32, 68
PIC, 54
PIT, 36
protected mode, 23

RAMFS, 79
Registers, 55
Ring 0, 51, 53
Ring 3, 51, 54

segment
    code, 50
    data, 50
shell, 91
storage
    ATA, 65
    read, 65
    write, 66
syscall, 57
system call, 54

terminal
    framebuffer, 71
text mode, 31
trap gate, 52
TSS, 51

VFS, 31, 77

zedit, 99
ZILO/SYS, 13